

Scala Notes



HP Conell

Scala è nato con lo scopo di ampliare Java, migliorando.

Scala è: leggera, ha una sintassi tipizzata

- Parallela, a seconda delle esigenze di usare programmazione OOP o funzionale.
- Un linguaggio scalabile, perfetto per Aggregato DSLs
- Stabile ed innovativo
- Compatibile con JAVA

REPL è una shell interattiva per scala, capace di compilare ed eseguire codice. Utile per test

VARIABILI → Immutabili, mutabili. (C'è Type Inference)

Uno statement viene eseguito senza return, per i suoi side effects, un'espressione ritorna un valore.

In Scala non sono necessarie parentesi grasse per espressioni a riga singola, le annotazioni dei tipi possono essere omesse, non servono traits & parentesi {}. Non c'è bisogno di RETURN

OO Basics

Object-orientation in Scala

- Classes and traits
 - Fields keep state
 - Methods provide operations (encapsulation)
 - Access modifiers declare visibility (information hiding)
- Singleton objects are first-class objects
- Inheritance
 - Single inheritance, i.e. extend exactly one superclass
 - Multiple traits can be mixed in

In Scala il costruttore primario è di default, se ne vuole definire uno si usa `def this`. I parametri di una classe non sono fields, possono essere usati nel body della classe ma non possono essere usati nell'initializer.

Essi possono diventare fields anticipando val o var al nome. I metodi forniscono operazioni in una classe: `def nome_meto(parametri): tipo_ritorno`

I metodi con un parametro possono usare la notazione `infer`. Usare `infer` per metodi con nomi autoesplicativi. Usare `infer` per unarg.

`Private` → solo classe / `Protected` → nell'istite e subclasses

42: **OBJECT** vengono usati per oggetti statici e singleton.

Un companion object è una classe o un'interfaccia che ha lo stesso nome di un object. Possono accedere ai membri privati dell'object.

CASE CLASS = - Non c'è bisogno di NEW per inizializzazione
- Crea un companion object (usare .apply)
- Crea toString, equals e hashCode, copy
- tutti i campi sono immutabili

Le classi **FINAL** → non estensibile, **SEALED** → solo nello stesso file

- SCALA ADVANCED

Una buona pratica è rendere le funzioni ricorsive **TAIL** → @tailrec nome funzione, buona come usare il tail dentro una funzione espone come local method.

Una **funzione parziale** è una funzione che restituisce una risposta solo per un sottoinsieme. Sono:

- unary operator (prendono un solo parametro)
- si applicano ad un sottoinsieme di valori
- possono avere una funzione **isDefinedAt** per capire se applicabile

Una **PF literal** è data da un blocco di case, bisogna stare attenti ai match con esaurienti.

SLIDING → raggiunge gli elementi di una collezione tramite una window.

FOLDLEFT → ^{right} riduce una collezione ad un valore singolo. Una funzione **binaria** si applica dal 1° agli altri elementi.

CUSTOM VALUE CLASS → Si deve estendere AnyVal. Bisogna: - abbiamo solo un parametro (deve essere val)

Le classi, interfacce e metodi possono essere **parametrically polymorphic**. Uno o più parametri possono essere passati tra parentesi quadre.

VARIANCE → Avviene per via della sostituzione. Nessuna relazione → **INVARIANT** → **Read-Write** → ^{senza segno}
Allineati → **Covariant** → **Read-only** → **can +**
Opposti → **Contravariant** → **Write-only** → **can -**

VALS sono **Covariant**, **VAR** sono **Contravariant** e **Contravariant**. Parametri delle funzioni **CONTRAVARIANT** mentre i return **COVARIANT**.

Essi possono essere gestiti dai **LOWER BOUNDS** con **>=** ^{deve essere solo 1 o 2} e **UPPER BOUNDS** **<=** ^{deve essere solo 1 o 2}.

Un tipo esistente può essere **ridefinito** usando dichiarazioni o definizioni, una dichiarazione lo aggiunge un nuovo membro e è **STRUCTURAL**. Le performance ne possono risentire.

Static duck typing

- The duck test: If it looks like a duck, swims like a duck, and quacks like a duck, then it probably is a duck
- Omitting the type to be refined you get a purely **structural type**:

```
type Closeable = { def close(): Unit }
```

```
scala> val c1: Closeable = new java.io.StringWriter  
c1: Closeable =
```

```
scala> val c2: Closeable = new Door  
c2: Closeable = Door@4cda3058
```

PHANTOM TYPES sono usati per mettere constraints ai type.

```
object Power {  
  sealed trait On extends Power  
  sealed trait Off extends Power  
}  
  
class LightSwitch[State <: Power] private {  
  def on[S >: State <: Power.On] = new LightSwitch[Power.On]  
  def off[S >: State <: Power.Off] = new LightSwitch[Power.Off]  
}  
  
object LightSwitch {  
  def on = new LightSwitch[Power.On]  
  def off = new LightSwitch[Power.Off]  
}
```

Rappresenta il
Phantom

Si può usare anche ::=

IMPLICIT CONVERSION sono metodi che prendono un tipo in input e ne restituiscono un'altro:

implicit def stringToInt(value: String): Int = Integer.parseInt value

Per essere applicati, deve essere in Scope (accessibile senza prefix)

Sono costituiti da tutti i membri degli oggetti associati. Deve essere solo uno

Type classes

- A type class is a polymorphic trait declaring an interface:

```
trait Bool[A] {  
  def bool(a: A): Boolean  
}
```

- A type class instance is an implicit value defining the interface for some type:

```
implicit val intBool = new Bool[Int] {  
  override def bool(n: Int): Boolean =  
    n != 0  
}
```

Type class glue

- A type class could be used standalone:

```
scala> intBool bool 0  
res0: Boolean = false
```

- Using an implicit parameter for the type class instance, supported types can be extended:

```
implicit class BoolOps[A](val a: A) extends AnyVal {  
  def bool(implicit instance: Bool[A]): Boolean =  
    instance bool a  
}
```

```
scala> 0.bool  
res0: Boolean = false
```

Using ClassTags

- To access an implicit ClassTag, use the classTag method
- The wrapped runtime class can be accessed via the runtimeClass method
- A ClassTag is also an extractor, i.e. defines an unapply method which is used in pattern matching:

```
import scala.reflect._  
  
def conforms[A : ClassTag](any: Any): Boolean =  
  (classTag[A] unapply any).isDefined  
  
scala> conforms[java.util.Date](new java.sql.Date(0))  
res0: Boolean = true  
  
scala> conforms[java.util.Date]("")  
res1: Boolean = false
```


Un DSL è un modo particolare di utilizzare un linguaggio ospite
per dare alla lingua ospite la sensazione di una lingua particolare.

Tecnicamente non c'è differenza tra DSL ed una libreria, tuttavia un DSL ha le seguenti proprietà:

- Comprensibile intuitivamente per gli esperti di dominio.
- Di alto livello e facile da usare
- Robusto